Python Engineering - Engineering & Programming Theory

Class 13 - Classes & Objects: modelling state and behaviour

The new idea in EX21 is not a physics equation but a way to represent systems. Object-oriented programming groups **state** and **behaviour** into one entity.

1. From separate variables to an entity

```
h = 100.0
v = 20.0
t = 0.0
g = 9.81
```

> These values are physically related: together they describe one state. A class lets us express that relationship explicitly.

2. State + behaviour

State (attributes)	Behaviour (methods)
h, v, t, g	specific_energy()
	state_vector()
	is_ascending()
	euler_step(dt)

3. Class vs object: blueprint and realization

VerticalState is a general definition. earth_state is one concrete realization. One class can create many independent objects.

4. What happens when an object is created?

```
1. Python creates a new VerticalState object
2. __init__ is called
3. self points to that new object
4. self.h = h stores the incoming height
5. self.v = v stores velocity
6. defaults are used unless overridden
7. the object reference is assigned to earth_state
```

5. Why is self necessary?

The same method code must operate on different objects. self identifies the particular object being used.

```
earth_state.specific_energy()  # self -> earth_state
moon_state.specific_energy()   # self -> moon_state
```

6. Object independence

> Earth and Moon objects may start with the same h and v but store different g values. Updating one object does not update the other.

7. Methods may inspect or mutate

specific_energy(), state_vector(), and is_ascending() inspect the state. euler_step(dt) changes stored h, v, and t.

8. Manual Earth example

Initial state:

$$h=100\ \text{m},\ v=20\ \text{m/s},\ t=0\ \text{s},\ g=9.81\ \text{m/s}^2$$

Specific energy:

$$E = g\,h + \tfrac{1}{2}\,v^2 = 1181\ \text{J/kg}$$

One Euler step, $dt=0.1$ s:

$$h_{new} = 102\ \text{m}$$
$$v_{new} = 19.019\ \text{m/s}$$
$$t_{new} = 0.1\ \text{s}$$

The object can store these new values internally rather than returning three unrelated variables.

9. Same model, different conditions

Earth and Moon use the same class implementation. Only g differs. The generic behaviour is defined once; objects provide their own state.

10. Class versus dictionary + functions

Dictionary + functions	Class + objects
Data and functions separate	State and behaviour grouped
State passed into functions	self accesses stored state
Good for simple structures	Natural for entities with identity/evolution
Less structure	More organization and reuse

11. OOP does not mean classes everywhere

Pure functions remain excellent for many calculations. Classes are useful when related data and operations naturally belong to an entity.

12. Connection to future projects

Future object	Possible state	Possible methods
Aircraft	position, velocity, mass, attitude	propagate(), forces()
Rocket	mass, altitude, velocity	thrust(), step()
Satellite	r, v, epoch	propagate_orbit()
IMU	bias, noise, rate	measure()
KalmanFilter	x, P, Q, R	predict(), update()

Final idea

A class models an entity. An object is one concrete entity. Attributes represent state and methods represent operations. EX21 reuses familiar vertical-motion physics so the focus stays on this new programming structure.